```
            cur_n = grid[x_,y_]
            s = get_substate(cur_n)                    4
            action_mean += s
    action_mean /= action_mean.sum()                   5
    return action_mean
```

- ***1*** Converts vectorized index j into grid coordinates [x,y], where [0,0] is top left
- ***2*** This will be the action mean vector that we will add to.
- ***3*** Two for loops allow us to find each of the 8 nearest neighbors of agent j.
- ***4*** Converts each neighbor's binary spin into an action vector
- ***5*** Normalizes the action vector to be a probability distribution

This function accepts an agent index, j (a single integer, the index based on the flattened grid) and returns that agent's eight nearest (surrounding) neighbors' mean action on the grid. We find the eight nearest neighbors by getting the agent's coordinates, such as [5,5], and then we just add every combination of [x,y] where $x,y \in \{0,1\}$. So we'll do [5,5] + [1,0] = [6,5] and [5,5] + [-1,1] = [4,6], etc.

These are all the additional functions we need for the 2D case. We'll re-use the `init_grid` function and `gen_params` functions from earlier. Let's initialize the grid and parameters.

```
>>> size = (10,10)
>>> J = np.prod(size)
>>> hid_layer = 10
>>> layers = [(2,hid_layer),(hid_layer,2)]
>>> params = gen_params(1,2*hid_layer+hid_layer*2)
>>> grid = init_grid(size=size)
>>> grid_ = grid.clone()
>>> grid__ = grid.clone()
>>> plt.imshow(grid)
>>> print(grid.sum())
```

We're starting with a 10 × 10 grid to make it run faster, but you should try playing with larger grid sizes. You can see in figure 9.21 that the spins are randomly distributed on the initial grid, so we hope that after we run our MARL algorithm it will look a lot more organized—we hope to see clusters of aligned electrons. We've reduced the hidden layer size to 10, to further